โครงงานพัฒนาเว็บแอปพลิเคชัน สำหรับบริหารจัดการระบบงาน

คะแนนเต็ม 30 คะแนนนำเสนอวันที่ 27–28 ตุลาคม 2569 กลุ่มละ 15 นาที

1 โจทย์โครงงาน

ให้นักศึกษาแต่ละคนเลือกพัฒนาเว็บแอปพลิเคชันสำหรับบริหารจัดการระบบงานที่สนใจ อะไรก็ได้ เช่น ระบบยืมคืนอุปกรณ์ ระบบจองห้อง ระบบคลังสินค้า ระบบแจ้งซ่อม หรือระบบจัดการกิจกรรม โดยมีขั้นตอนการทำงานชัดเจน ผู้ใช้หลายบทบาท และสาธิตการใช้งานได้จริง

ใช้ Node.js และ Express.js ร่วมกับการสร้าง Middleware และระบบจัดการฐานข้อมูล PostgreSQL พร้อมกับให้มีระบบยืนยันตัวตนการใช้งาน พร้อมการกำหนดสิทธิ์ด้วยเทคนิค ACL และมีการจัดการ Role และ Permission ผ่านหน้าเว็บ ในเว็บต้องจัดให้มีการบันทึก Log เพื่อดูย้อนหลังการใช้งานของผู้ใช้แต่ละคนว่าทำงานอะไรส่วนไหนของเว็บบ้างวันเวลาใด

2 วัตถุประสงค์

- พัฒนาเว็บแอปพลิเคชันที่เชื่อมต่อ PostgreSQL ได้
- ใช้ Middleware ตรวจสอบการเข้าสู่ระบบ สิทธิ์ และบันทึก Log ได้
- พัฒนา Authentication และ Authorization โดยจัดการ Role, Permission และ ACL ผ่านเว็บได้
- ออกแบบ Access Matrix และ ER Diagram ให้ตรงกับระบบจริงได้
- ทดสอบสิทธิ์การเข้าถึงและนำเสนอผลการทำงานพร้อมหลักฐานได้

3 การทำงานหลักและเทคโนโลยี

- ใช้ Node.js และ Express.js ฝั่ง Server และ PostgreSQL จัดเก็บข้อมูล
- มีการเพิ่ม อ่าน แก้ไข และลบข้อมูลตามความเหมาะสมของระบบงาน
- มีกระบวนการหลักอย่างน้อย 1 กระบวนการ เช่น ส่งคำขอ ⟶ พิจารณา ⟶ อนุมัติหรือปฏิเสธ ⟶ ตรวจสอบสถานะ
- ตรวจสอบข้อมูลฝั่ง Server และจัดการข้อผิดพลาดอย่างเหมาะสม

4 การยืนยันตัวตนและการจัดการสิทธิ์

4.1 Authentication และรหัสผ่าน

- มีระบบเข้าสู่ระบบและออกจากระบบ โดยใช้ Session หรือ Token และอธิบายวิธีที่เลือกได้
- จัดเก็บรหัสผ่านด้วย Hash พร้อม Salt เช่น bcrypt หรือ Argon2
  ห้ามเก็บเป็นข้อความปกติหรือใช้การเข้ารหัสที่ถอดกลับเป็นรหัสผ่านเดิมได้
- ตรวจสอบรหัสผ่านด้วยฟังก์ชันของไลบรารีที่เลือก ไม่แสดง Password Hash บนเว็บ
  และไม่บันทึกรหัสผ่านลงใน Log
- ผู้ที่ยังไม่เข้าสู่ระบบต้องเข้าถึงหน้าและ API ที่ได้รับการป้องกันไม่ได้
  หลังออกจากระบบต้องใช้สถานะเดิมเพื่อเข้าถึงข้อมูลไม่ได้

4.2 การจัดการ Role ผ่านเว็บ

- เริ่มต้นด้วยอย่างน้อย 4 Role รวม Admin โดยแต่ละบทบาทมีหน้าที่และสิทธิ์แตกต่างกันอย่างมีเหตุผล
- Admin ใช้งานทุกฟังก์ชัน รวมถึงฟังก์ชันที่เพิ่มภายหลัง และจัดการผู้ใช้ บทบาท และสิทธิ์ได้
- Admin เพิ่ม แก้ไข และลบ Role รวมทั้งกำหนดหรือเปลี่ยน Role ของผู้ใช้ผ่านเว็บได้ โดยไม่ต้องแก้ Source Code
- ป้องกันการลบ Role ที่ยังมีผู้ใช้ และป้องกันการเปลี่ยนแปลงที่ทำให้ไม่มี Admin เหลืออยู่

4.3 การจัดการ Permission และ ACL ผ่านเว็บ

- Admin เพิ่ม แก้ไข และลบรายการ Permission ผ่านเว็บได้ เช่น equipment.read, equipment.create และ equipment.update
- Admin กำหนดและยกเลิก Permission ให้ Role ผ่านหน้า ACL รูปแบบตาราง Role × Permission ได้
- เก็บ Role, Permission และความสัมพันธ์ใน PostgreSQL โดย Middleware
  ตรวจสอบสิทธิ์จากข้อมูลดังกล่าว

การตรวจสอบสิทธิ์และส่วน Admin

- ตรวจสอบสิทธิ์ฝั่ง Server ทุก Route และ API ที่ได้รับการป้องกัน
  การซ่อนเมนูหรือปุ่มเพียงอย่างเดียวไม่เพียงพอ
- กำหนด ACL ให้ชัดเจนว่าบทบาทใดเข้าถึงทรัพยากรและกระทำการใดได้ เช่น ดู เพิ่ม แก้ไข ลบ และอนุมัติ
- เมื่อสิทธิ์ขึ้นกับเจ้าของข้อมูล ต้องตรวจเงื่อนไขนั้นด้วย เช่น ผู้ใช้แก้ไขได้เฉพาะรายการของตนเอง
- การเพิ่ม Role หรือเปลี่ยนการกำหนดสิทธิ์ต้องมีผลกับฟังก์ชันที่ระบบรองรับ โดยไม่ต้องแก้ Source Code หรือเริ่ม Server ใหม่
- Permission ใหม่ต้องเชื่อมกับการตรวจสอบสิทธิ์ของฟังก์ชันหรือ API ที่เกี่ยวข้อง การเพิ่มชื่อ Permission ในฐานข้อมูลเพียงอย่างเดียวไม่ถือว่ามีการป้องกันแล้ว

5.1 การแยกส่วน Admin ทาง Logical

แยกหน้า เมนู กลุ่ม Route และการตรวจสอบสิทธิ์ของส่วน Admin ออกจากเว็บทั่วไป เช่น /app/... สำหรับผู้ใช้ทั่วไป /admin/... สำหรับ Admin และ /api/admin/... สำหรับ API ของ Admin โดยใช้ Server และฐานข้อมูลร่วมกันได้

ผู้ที่ไม่มีสิทธิ์ต้องเข้าถึงส่วน Admin ไม่ได้ แม้พิมพ์ URL หรือเรียก API โดยตรง

5.2 Access Matrix

จัดทำตาราง Role × ทรัพยากรหรือฟังก์ชัน × การกระทำ ให้ครบและตรงกับ ACL จริง หากมีเงื่อนไขเฉพาะข้อมูลของตนเองหรือหน่วยงานของตนต้องระบุให้ชัดเจน ตัวอย่างต่อไปนี้ต้องปรับตามระบบ ของแต่ละกลุ่ม

ฟังก์ชัน	Admin	ผู้จัดการ	เจ้าหน้าที่	ผู้ใช้ทั่วไป
คำขอ	ทุกสิทธิ์	ดูทั้งหมด อนุมัติ	ดูและปรับสถานะที่รับผิดชอบ	เพิ่มและดูของตน
อุปกรณ์	ทุกสิทธิ์	ดู	ดู เพิ่ม แก้ไข	ดู
ผู้ใช้และสิทธิ์	ทุกสิทธิ์	ไม่มี	ไม่มี	ไม่มี
Log	ทุกสิทธิ์	ดูตามกำหนด	ไม่มี	ไม่มี

การบันทึก Log และดูย้อนหลัง

- จัดเก็บ Log ใน PostgreSQL และยังเรียกดูได้หลังเริ่ม Server ใหม่
- บันทึกการเข้าสู่ระบบสำเร็จและล้มเหลว การออกจากระบบ การเพิ่ม แก้ไข หรือลบข้อมูลสำคัญ การเปลี่ยน Role หรือ Permission และการเข้าถึงที่ถูกปฏิเสธ
- Log มีวันเวลา ผู้กระทำหรือชื่อบัญชีที่พยายามเข้าสู่ระบบ เหตุการณ์ ทรัพยากรหรือรายการที่เกี่ยวข้อง และผลการทำงาน
- มีหน้าดู Log ย้อนหลัง ค้นหาหรือกรองตามช่วงเวลา ผู้ใช้ และประเภทเหตุการณ์ได้
- จำกัดสิทธิ์การดู Log และไม่บันทึกรหัสผ่าน Session ID หรือ Token ลงใน Log

7 ฐานข้อมูลและ ER Diagram

- ออกแบบฐานข้อมูลให้รองรับระบบงาน ผู้ใช้ Role, Permission, ACL และ Log
- กำหนด Primary Key, Foreign Key และข้อจำกัดข้อมูลอย่างเหมาะสม

• จัดทำ ER Diagram ที่ตรงกับฐานข้อมูลจริง แสดง Entity, Attribute และความสัมพันธ์พร้อม Cardinality

• อธิบายความสัมพันธ์ระหว่างผู้ใช้ บทบาท และสิทธิ์ รวมถึงการเชื่อมโยงกับข้อมูลระบบงานได้

8 สิ่งที่ต้องส่ง

• Source Code พร้อมวิธีติดตั้ง ตั้งค่า และเริ่มใช้งาน

• ไฟล์ SQL หรือ Migration สำหรับสร้างฐานข้อมูลและข้อมูลตัวอย่าง

• บัญชีทดสอบสำหรับทุก Role

• ER Diagram และ Access Matrix

• สไลด์ที่อธิบายปัญหา ขอบเขต กระบวนการ สถาปัตยกรรม Middleware บทบาทและสิทธิ์ ฐานข้อมูล และผลการทดสอบ

ไม่ใส่รหัสผ่านฐานข้อมูลจริงหรือข้อมูลลับใน Source Code ที่ส่ง ให้จัดเตรียมไฟล์ตัวอย่างการตั้งค่าแทน

9 การอัปโหลดไฟล์หรือภาพ

ระบบต้องมีการอัปโหลดไฟล์หรือภาพอย่างน้อย 1 ส่วนที่สัมพันธ์กับระบบงาน เช่น ภาพอุปกรณ์ เอกสารประกอบคำขอ ภาพแจ้งซ่อม หรือไฟล์แนบกิจกรรม และสามารถเปิดดูหรือดาวน์โหลดตามสิทธิ์ได้จริง

• มีแบบฟอร์มอัปโหลดผ่านหน้าเว็บ และเชื่อมโยงไฟล์กับรายการข้อมูลที่เกี่ยวข้อง

• กำหนดชนิดไฟล์ที่อนุญาตและขนาดสูงสุดให้ชัดเจน ตรวจสอบทั้งขนาดและชนิดที่ฝั่ง Server ไม่อาศัยนามสกุลหรือการตรวจใน Browser เพียงอย่างเดียว

• ตั้งชื่อไฟล์จัดเก็บให้ไม่ชนกัน และป้องกันชื่อหรือเส้นทางไฟล์ที่ไม่ปลอดภัย

• เก็บข้อมูลอ้างอิงไฟล์ใน PostgreSQL เช่น ชื่อเดิม ชื่อหรือเส้นทางจัดเก็บ ชนิด ขนาด ผู้อัปโหลด วันเวลา และรายการที่เกี่ยวข้อง โดยเก็บตัวไฟล์ในพื้นที่จัดเก็บที่เหมาะสมได้

• กำหนด Permission สำหรับอัปโหลด ดูหรือดาวน์โหลด และลบไฟล์ตามขอบเขตระบบ พร้อมระบุใน Access Matrix

• ตรวจสอบสิทธิ์ฝั่ง Server ทั้งตอนอัปโหลดและตอนเปิดหรือดาวน์โหลด หากเป็นไฟล์ที่จำกัดสิทธิ์ ต้องไม่เปิดให้เข้าถึงโดยตรงผ่าน URL สาธารณะ

• บันทึก Log การอัปโหลดและลบไฟล์ รวมถึงการเข้าถึงไฟล์ที่ถูกปฏิเสธ และแสดงข้อความข้อผิดพลาดที่เข้าใจได้

หลักฐานการทดสอบไฟล์

• อัปโหลดไฟล์ที่ถูกต้อง แล้วเปิดดูภาพหรือดาวน์โหลดไฟล์ได้

• อัปโหลดไฟล์ผิดชนิดหรือเกินขนาด แล้วระบบปฏิเสธ

• ใช้บัญชีที่ไม่มีสิทธิ์เรียก URL หรือ API ของไฟล์ที่จำกัดสิทธิ์ แล้วระบบปฏิเสธ

• แสดงข้อมูลไฟล์ที่เชื่อมกับรายการงานและ Log ที่เกี่ยวข้อง

10 การนำเสนอและสาธิตระบบ

นำเสนอวันที่ 27–28 ตุลาคม 2569 กลุ่มละ 15 นาที แบ่งเป็นนำเสนอและสาธิต 12 นาที และตอบคำถาม 3 นาที

ประเด็นที่ต้องนำเสนอ

• ปัญหาที่ต้องการแก้ ผู้ใช้เป้าหมาย ขอบเขต และกระบวนการหลัก

• สถาปัตยกรรมและลำดับการทำงานของ Middleware

• ER Diagram และความสัมพันธ์ของข้อมูล

• Access Matrix และการตรวจสอบสิทธิ์ตาม Role, Permission และ ACL

• ผลการทดสอบ ปัญหาที่พบ และข้อจำกัดของระบบ

หลักฐานที่ต้องสาธิตจากระบบจริง

1. กระบวนการหลักตั้งแต่เริ่มต้นจนเสร็จสิ้น

2. เข้าสู่ระบบและแสดงความแตกต่างของสิทธิ์ทั้ง 4 Role

3. Admin เข้าถึงทุกฟังก์ชันและจัดการผู้ใช้ บทบาท และสิทธิ์

4. ผู้ที่ไม่มีสิทธิ์เข้าหน้า Admin หรือเรียก API โดยตรง แล้วถูกปฏิเสธ

5. เพิ่ม Role ใหม่ผ่านเว็บและกำหนดให้บัญชีทดสอบ

6. เพิ่มรายการ Permission ผ่านเว็บ เชื่อมกับฟังก์ชันที่รองรับ และกำหนดให้ Role
เพื่อแสดงว่าบัญชีทดสอบใช้งานได้

7. ยกเลิก Permission แล้วแสดงว่าบัญชีเดิมถูกปฏิเสธ ทั้งหน้าเว็บและ API โดยไม่เริ่ม Server ใหม่

8. อัปโหลดไฟล์หรือภาพ เปิดดูหรือดาวน์โหลด และแสดงการปฏิเสธไฟล์ที่ไม่ผ่านเงื่อนไขหรือการเข้าถึงที่ไม่มีสิทธิ์

9. เรียกดูและกรอง Log ของการเพิ่ม Role การเปลี่ยนสิทธิ์ การอัปโหลด และการเข้าถึงที่ถูกปฏิเสธ

11 เกณฑ์การให้คะแนนรวม 30 คะแนน

ประเด็นประเมิน	คะแนน	เกณฑ์สำหรับคะแนนเต็ม
1 ระบบงานและการอัปโหลด	5	ระบบงานครบและถูกต้อง 3 คะแนน อัปโหลดไฟล์หรือภาพเชื่อมกับรายการ และเปิดดูหรือดาวน์โหลดได้ 2 คะแนน
2 Node.js Express.js และ Middleware	4	จัด Route เหมาะสม ใช้ Middleware ตรวจการเข้าสู่ระบบ สิทธิ์ และ Log ตรวจข้อมูล รวมถึงชนิดและขนาดไฟล์ และจัดการข้อผิดพลาดได้
3 PostgreSQL และ ER Diagram	4	ฐานข้อมูลรองรับงานจริง Key และความสัมพันธ์ถูกต้อง ER Diagram ตรงกับระบบ และอธิบายได้
4 Authentication	4	เข้าสู่ระบบและออกจากระบบได้ ใช้ Hash พร้อม Salt ตรวจรหัสผ่านและจัดการ Session หรือ Token ถูกต้อง
5 Authorization ACL และส่วน Admin	5	มีอย่างน้อย 4 Role Admin ทำได้ทุกฟังก์ชัน จัดการ Role Permission และ ACL ผ่านเว็บได้ เปลี่ยนสิทธิ์มีผลจริง ตรวจฝั่ง Server รวมสิทธิ์ไฟล์ และป้องกัน URL หรือ API โดยตรง
6 Access Matrix	2	ตารางครบ ระบุเงื่อนไขชัดเจน ตรงกับ ACL ที่กำหนดผ่านเว็บและผลทดสอบจริง
7 Log และการดูย้อนหลัง	3	บันทึกเหตุการณ์ครบ เก็บถาวร ค้นหาและกรองได้ จำกัดสิทธิ์ดู และไม่บันทึกข้อมูลลับ
8 การนำเสนอและตอบคำถาม	3	อธิบายชัด สาธิตครบ อยู่ในเวลา ตอบคำถามได้ และสมาชิกทุกคนมีส่วนร่วม
รวม	30	

แนวทางให้คะแนน

ให้คะแนนตามความครบถ้วน ความถูกต้อง และหลักฐานจากระบบจริงในแต่ละประเด็น

ทำได้ครบได้รับคะแนนเต็ม ทำได้บางส่วนได้รับคะแนนตามผลงาน

ส่วนที่มีเฉพาะคำอธิบายหรือภาพในสไลด์แต่สาธิตไม่ได้ จะไม่ได้คะแนนส่วนการทำงานจริงที่เกี่ยวข้อง